

Selecting Tools & Frameworks for Capstone Infra

Navigating the tooling ecosystem for your infrastructure project doesn't have to be overwhelming. The right stack balances productivity with scalability while matching your specific domain, infrastructure requirements, metrics, and budget constraints.



Why Tool Selection Matters

Developer Velocity

Your tool choices directly impact how quickly your team can iterate, test, and deploy. The right frameworks accelerate development while preventing technical debt.

Infrastructure Costs

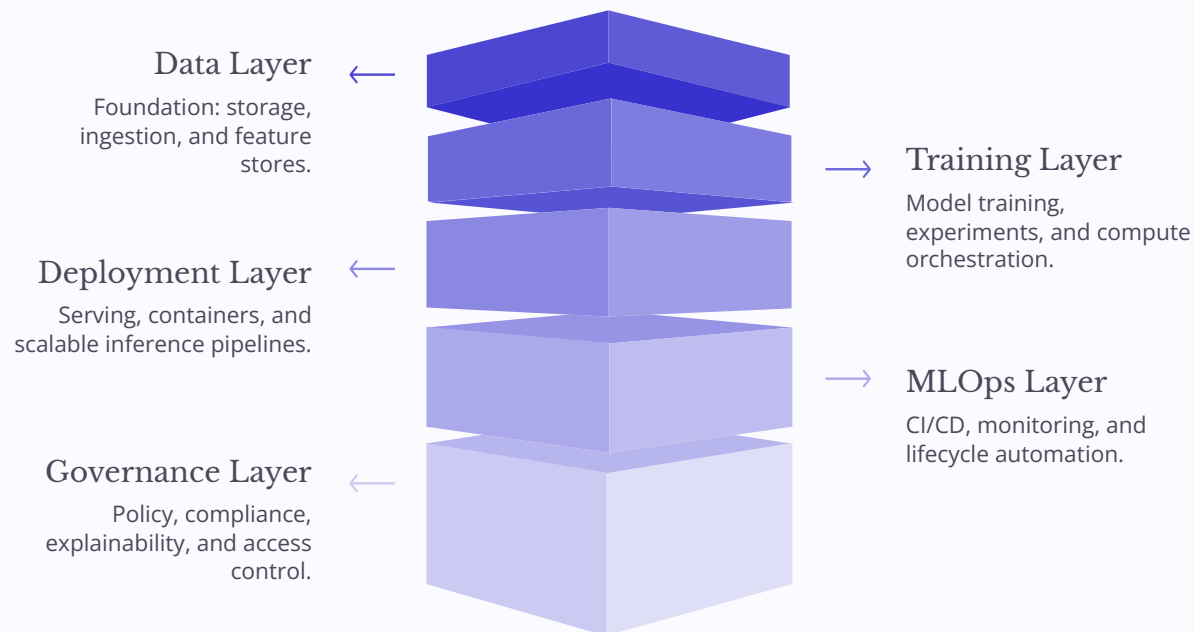
Tools influence hosting requirements, compute resources, and scaling patterns—affecting both short-term project expenses and long-term TCO.

Portability & Lock-in

Wrong choices lead to vendor lock-in and wasted effort migrating between incompatible systems. Portable stacks preserve future flexibility.

Remember: Capstone success demands practical, not exotic, tooling decisions.

Key Tooling Layers



A well-designed infrastructure stack addresses each layer with complementary tools that work together seamlessly. Your capstone should consider which layers are most critical to your specific project goals.

Data Layer Tools

Storage Solutions

AWS S3, GCP Cloud Storage, Azure Blob Storage

Feature Stores

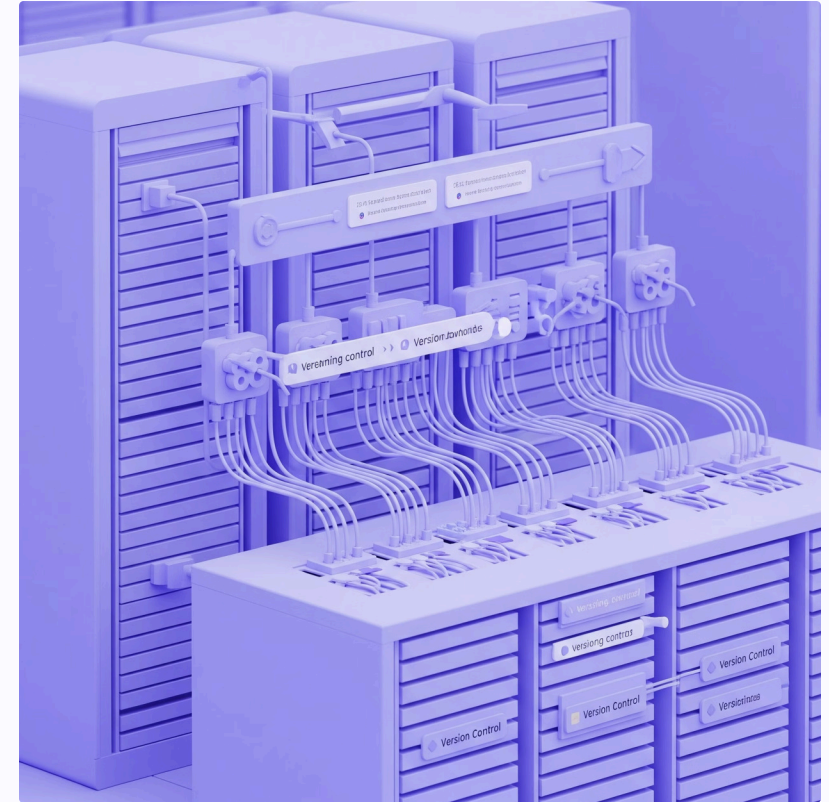
Feast, Tecton, Databricks Feature Store

Streaming Platforms

Kafka, Flink, Google Pub/Sub

Data Versioning

DVC, LakeFS



Training Frameworks

PyTorch

Flexibility and research-driven development make it ideal for LLMs, computer vision, and NLP projects.

- Dynamic computation graph
- Pythonic API
- Strong research community

TensorFlow + Keras

Production stability and TPU optimization for enterprise-grade deployments.

- Static graph performance
- TF Extended ecosystem
- Google Cloud integration

JAX

High-performance computing with TPU acceleration for research-intensive projects.

- Functional approach
- NumPy-like API
- XLA compilation

Accelerator libraries like [DeepSpeed](#), [Horovod](#), and [Ray Train](#) can enhance any framework's distributed training capabilities.

Deployment Frameworks

Containerization & Orchestration



Docker

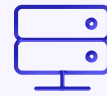
Package models with dependencies for consistent deployment across environments



Kubernetes

Orchestrate containers across clusters (EKS, GKE, AKS)

Serving Infrastructure



Inference Servers

TorchServe, Triton Inference Server, FastAPI



Edge Runtimes

TensorRT, ONNX Runtime, CoreML

Your deployment architecture should balance latency requirements, throughput needs, and infrastructure costs.

MLOps & Pipeline Tools



Experiment Tracking

MLflow, Weights & Biases



Workflow Orchestration

Airflow, Argo, Kubeflow Pipelines



CI/CD

GitHub Actions, GitLab CI, Jenkins



Monitoring

Prometheus, Grafana, EvidentlyAI



Governance & Compliance Tools

Building Trust Through Transparency

Governance tools ensure your models are explainable, auditable, and secure—critical for capstone projects that may involve sensitive data or high-stakes decisions.

- ❏ For healthcare, financial, or government-related capstones, governance tools aren't optional—they're essential requirements.

Explainability

SHAP, LIME, Captum

Model Cards

Hugging Face, MLflow registry

Audit Logging

ELK stack, cloud-native logging

Security

IAM, Vault, KMS encryption

Example Stacks (by Domain)

NLP RAG System

- PyTorch for model training
- FAISS for vector similarity search
- FastAPI for serving endpoints
- MLflow for experiment tracking

Vision Edge AI

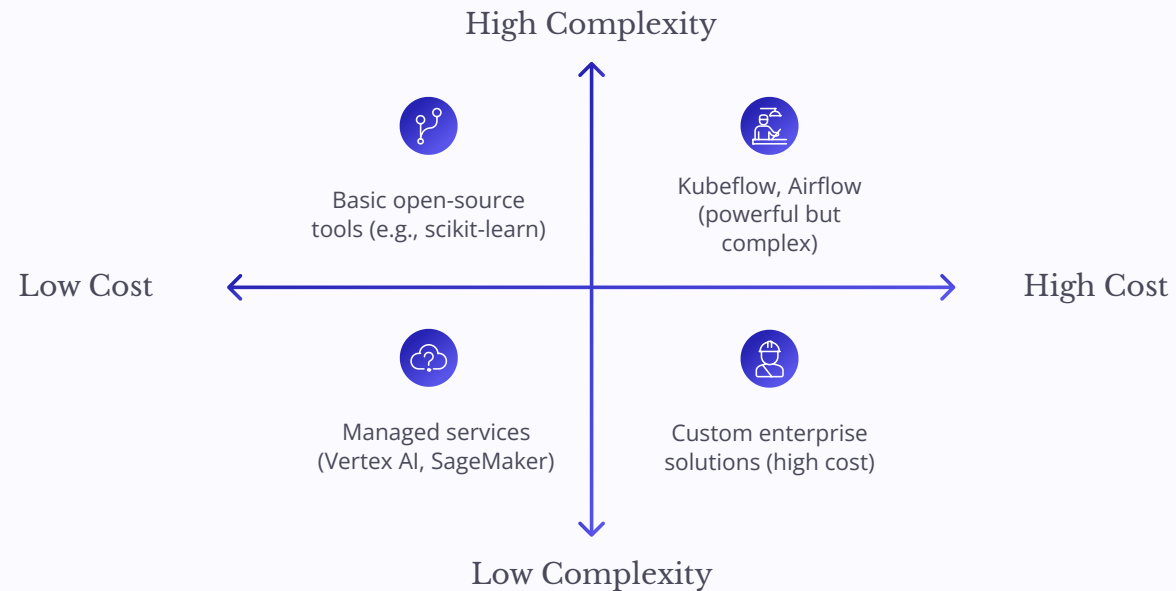
- TensorFlow Lite for model optimization
- ONNX Runtime for cross-platform inference
- Kubernetes Edge for distributed deployment

Generative AI

- PyTorch for model architecture
- DeepSpeed for training optimization
- Triton Server for high-throughput serving
- Weights & Biases for visualization

Each domain has typical tooling patterns that have proven effective in production environments.

Cost & Complexity Considerations



For capstone projects, the sweet spot is typically in the lower-cost quadrants, with complexity adjusted based on team experience and project timeline.

Balance DIY flexibility against managed service speed—and always avoid overengineering.

Case Study: Healthcare AI Capstone



Data Layer

LakeFS for versioning +
custom de-identification
pipeline to ensure HIPAA
compliance

1

Deployment Layer

Docker containers
orchestrated by Kubernetes
for scalable inference

3

2

Training Layer

TensorFlow on GCP TPU
pods for accelerated training
of medical imaging models

4

Governance Layer

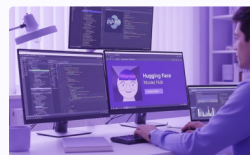
EvidentlyAI for drift detection
+ SHAP for model
interpretability

Case Study: Startup-Style NLP Capstone



Data Layer

S3 for document storage + Kafka for real-time text ingestion pipeline



Training Layer

PyTorch + Hugging Face Transformers for fine-tuning pre-trained language models

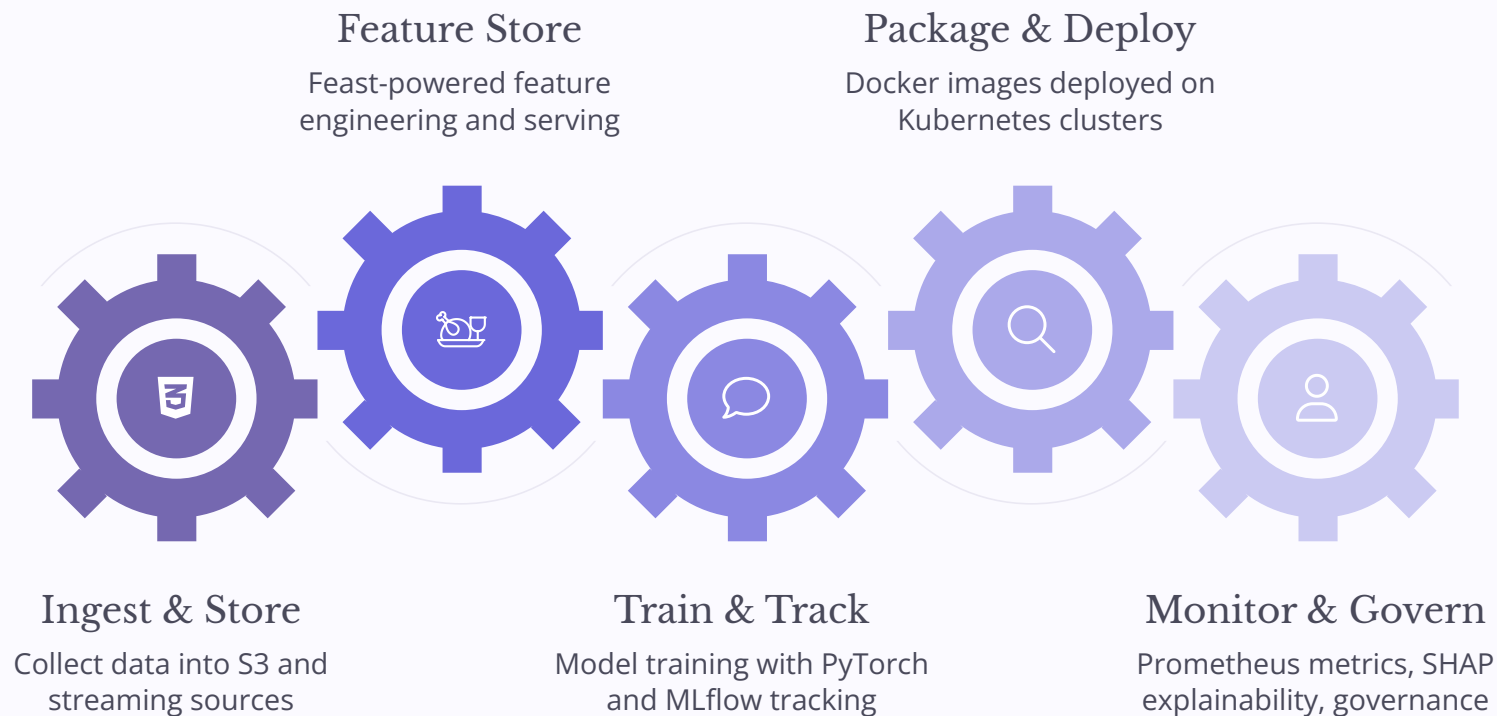


Deployment Layer

FastAPI for REST endpoints + Triton Server in Kubernetes for high-throughput inference

Cost optimization: Spot instance GPUs for training to stay under budget constraints

The Infrastructure Stack Visualized



Stack choice balances flexibility, scalability, compliance, and cost.

Key Lessons



Developer Speed

Tools define both development velocity and infrastructure efficiency. Choose frameworks that minimize friction for your team's experience level.



Integration Over Novelty

Avoid tool sprawl by prioritizing integration capabilities over novel features. A cohesive stack outperforms a collection of best-in-class but disconnected tools.



Domain Alignment

Pick frameworks that match your specific domain requirements and infrastructure metrics. Not all tools work equally well for all problem types.



Lean But Realistic

Capstone projects are proof-of-concepts. Keep your stack lean but industry-realistic to demonstrate both technical skill and practical judgment.



Discussion: The DIY vs. Managed Services Tradeoff

Open-Source DIY Tools

- Flexible customization
- No direct costs
- Full control over implementation
- Higher maintenance burden
- Steeper learning curve

Cloud-Managed Services

- Faster time-to-implementation
- Reduced operational overhead
- Enterprise-grade reliability
- Higher direct costs
- Potential vendor lock-in

Which would you choose for your Capstone project, and why?